

Patrick Murray

Professor Rivas

Database Management

24 April 2026

Part A:

```
SQL : SELECT grantee, table_name, privilege_type
      FROM information_schema.role_table_grants
      WHERE grantee IN ('advisor_role', 'registrar_role')
      AND table_name IN ('lab8_students', 'lab8_courses', 'lab8_enrollments')
      ORDER BY grantee, table_name, privilege_type;
```

Output: "grantee"	"table_name"	"privilege_type"
"advisor_role"	"lab8_courses"	"SELECT"
"advisor_role"	"lab8_enrollments"	"SELECT"
"advisor_role"	"lab8_students"	"SELECT"
"registrar_role"	"lab8_courses"	"SELECT"
"registrar_role"	"lab8_enrollments"	"INSERT"
"registrar_role"	"lab8_enrollments"	"SELECT"
"registrar_role"	"lab8_enrollments"	"UPDATE"
"registrar_role"	"lab8_students"	"SELECT"

Explanation: The advisor role only has access to selecting the three main tables of information. The registrar role on the other hand can do all the advisor can do plus inserting and updating the enrollments table.

Part B:

Procedure Creation Statement:

```
CREATE OR REPLACE PROCEDURE register_student(  
    procStudent_id INT,  
    procCourse_id TEXT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- 1. Check if student exists  
    IF NOT EXISTS (  
        SELECT 1 FROM lab8_students s WHERE s.student_id = procStudent_id  
    ) THEN  
        RAISE NOTICE 'Student does not exist';  
        RETURN;  
    END IF;  
  
    -- 2. Check if course exists  
    IF NOT EXISTS (
```

```
        SELECT 1 FROM lab8_courses c WHERE c.course_id = procCourse_id
    ) THEN

        RAISE NOTICE 'Course does not exist';

        RETURN;

    END IF;

-- 3. Check if already enrolled

    IF EXISTS (

        SELECT 1 FROM lab8_enrollments e

        WHERE e.student_id = procStudent_id

        AND e.course_id = procCourse_id

    ) THEN

        RAISE NOTICE 'Student is already enrolled';

        RETURN;

    END IF;

-- 4. Check if course is full

    IF (

        SELECT COUNT(*) FROM lab8_enrollments e WHERE e.course_id = procCourse_id

    ) >= (

        SELECT c.capacity FROM lab8_courses c WHERE c.course_id = procCourse_id)

        THEN

            RAISE NOTICE 'No room in the class';
```

```
RETURN;  
END IF;  
  
-- 5. Insert enrollment  
  
INSERT INTO lab8_enrollments(student_id, course_id)  
VALUES (procStudent_id, procCourse_id);  
  
RAISE NOTICE 'Student successfully enrolled';  
  
END;  
  
$$;
```

Query 1: CALL register_student(4,'MATH201');

Query 1 Result:NOTICE: Student is already enrolled

CALL

Query returned successfully in 92 msec.

Query 2: CALL register_student(4,'MATH201');

Query 2 Result: NOTICE: Student is already enrolled

CALL

Query returned successfully in 75 msec.

Query 3: CALL register_student(7, 'ART100');

Query 3 Result: NOTICE: No room in the class

CALL

Query returned successfully in 76 msec.

Explanation: This is a good example of logic that can be centralized into the database because you can repeatedly call this block of code whenever you need to without really risking it doing something bad to your database since the stored code in the procedure already has that covered.

Part C:

Function:

```
CREATE OR REPLACE FUNCTION audit_enrollment_insert()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO lab8_enrollment_audit(student_id, course_id, action_type)
    VALUES (NEW.student_id, NEW.course_id, 'INSERT');
    RETURN NEW;
END;
$$;
```

Trigger:

```
CREATE TRIGGER audit_enroll
    AFTER INSERT ON lab8_enrollments
    FOR EACH ROW
    EXECUTE FUNCTION audit_enrollment_insert();
```

Call and Test:

```
CALL register_student(2, 'MATH201');
```

Select and Output:

```
SELECT * FROM lab8_enrollment_audit
```

1	"ENROLL"	1	"CS101"	"2026-04-24 13:33:54.741533"
2	"ENROLL"	2	"CS101"	"2026-04-24 13:33:54.741533"
3	"ENROLL"	3	"CS101"	"2026-04-24 13:33:54.741533"
4	"ENROLL"	4	"MATH201"	"2026-04-24 13:33:54.741533"
5	"ENROLL"	5	"MATH201"	"2026-04-24 13:33:54.741533"
6	"ENROLL"	6	"ENG150"	"2026-04-24 13:33:54.741533"
7	"ENROLL"	7	"BIO110"	"2026-04-24 13:33:54.741533"
8	"ENROLL"	8	"BIO110"	"2026-04-24 13:33:54.741533"
9	"ENROLL"	9	"ART100"	"2026-04-24 13:33:54.741533"
10	"DROP"	3	"CS101"	"2026-04-24 13:33:54.741533"
11	"DROP"	6	"ENG150"	"2026-04-24 13:33:54.741533"
12	"INSERT"	2	"MATH201"	"2026-04-24 18:09:19.074498"

Explanation: This logic is a better fit than a query because this runs automatically when you do insertions into enrollment instead of having to type it manually. Overall, this allows you to almost automate a lot of stuff that would ordinarily be tedious but with triggers you can have it run automatically.

Part D:

Query 1: SELECT * FROM lab8_enrollments;

Query 1 Output:

1	"CS101"	"2026-04-24 13:33:54.741533"
2	"CS101"	"2026-04-24 13:33:54.741533"
3	"CS101"	"2026-04-24 13:33:54.741533"
4	"MATH201"	"2026-04-24 13:33:54.741533"
5	"MATH201"	"2026-04-24 13:33:54.741533"
6	"ENG150"	"2026-04-24 13:33:54.741533"
7	"BIO110"	"2026-04-24 13:33:54.741533"
8	"BIO110"	"2026-04-24 13:33:54.741533"
9	"ART100"	"2026-04-24 13:33:54.741533"
1	"MATH201"	"2026-04-24 16:09:45.741703"
2	"MATH201"	"2026-04-24 18:09:19.074498"

Query 2: SELECT course_id, title, capacity, enrolled_count, capacity-enrolled_count AS
seats_left FROM lab8_courses;

Query 2 Output:

"CS101"	"Intro to Computer Science"	30	3	27
"ENG150"	"English Literature"	20	1	19

"BIO110"	"General Biology"	40	2	38
"ART100"	"Foundations of Art"	1	1	0
"MATH201"	"Calculus II"	25	4	21

Query 3: SELECT * FROM lab8_enrollment_audit

ORDER BY action_time DESC;

Query 3 Output:

12	"INSERT"	2	"MATH201"	"2026-04-24 18:09:19.074498"
2	"ENROLL"	2	"CS101"	"2026-04-24 13:33:54.741533"
3	"ENROLL"	3	"CS101"	"2026-04-24 13:33:54.741533"
4	"ENROLL"	4	"MATH201"	"2026-04-24 13:33:54.741533"
5	"ENROLL"	5	"MATH201"	"2026-04-24 13:33:54.741533"
6	"ENROLL"	6	"ENG150"	"2026-04-24 13:33:54.741533"
1	"ENROLL"	1	"CS101"	"2026-04-24 13:33:54.741533"
8	"ENROLL"	8	"BIO110"	"2026-04-24 13:33:54.741533"
9	"ENROLL"	9	"ART100"	"2026-04-24 13:33:54.741533"
10	"DROP"	3	"CS101"	"2026-04-24 13:33:54.741533"
11	"DROP"	6	"ENG150"	"2026-04-24 13:33:54.741533"
7	"ENROLL"	7	"BIO110"	"2026-04-24 13:33:54.741533"

Query 4: SELECT grantee, table_name, privilege_type

FROM information_schema.role_table_grants

WHERE grantee IN ('advisor_role', 'registrar_role')


```
AND table_name IN ('lab8_students', 'lab8_courses', 'lab8_enrollments')

ORDER BY grantee, table_name, privilege_type;
```

Query 4 Output:

```
"advisor_role" "lab8_courses"      "SELECT"
"advisor_role" "lab8_enrollments"   "SELECT"
"advisor_role" "lab8_students"      "SELECT"
"registrar_role"      "lab8_courses"      "SELECT"
"registrar_role"      "lab8_enrollments"   "INSERT"
"registrar_role"      "lab8_enrollments"   "SELECT"
"registrar_role"      "lab8_enrollments"   "UPDATE"
"registrar_role"      "lab8_students"      "SELECT"
```

Part E:

1: Authentication is the verification of someone's identity while authorization is the privileges that that person has.

2: Least privileges is a good security measure because if you get someone with power who becomes disgruntled or someone who manages to get into the system, they can do damage to the database if they have a lot of privileges. If the privileges are given out sparingly however, this ensures that only a little bit of damage will be done if something like this arises.

3: They're both chunks of code but store procedures have to be called while triggers run automatically.

4: If you have too many triggers, they all are going to be doing stuff automatically which can make it hard to track if there's an issue with one of many automatically running triggers each doing their own tasks.

Github: <https://github.com/Not-Patrick1/DatabaseLabs/tree/main/Lab8>